

Práctica 1: Expresiones regulares

Autómatas y Lenguajes · Curso 2026/27

Duración orientativa: 1 semana. **Entrega:** fecha indicada en Moodle. **Calificación:** 10 puntos. El peso en la nota de prácticas será el indicado por el profesorado.

Objetivos y material

Diseñar expresiones regulares para reconocer lenguajes, validar formatos, extraer información y realizar sustituciones sencillas con el módulo `re` de Python. Repasa la presentación de expresiones regulares de la asignatura antes de comenzar.

Se facilitan `regular_expressions.py` y `test_p1.py`. Completa las cadenas RE1 a RE8 y las cadenas de sustitución SUB7 y SUB8. RE0 es un ejemplo resuelto. No es necesario programar funciones auxiliares: toda la solución debe quedar en los patrones y las cadenas de sustitución. Puedes definir fragmentos auxiliares de patrones.

Cómo trabajar

- Utiliza Python 3 y, preferentemente, cadenas crudas: `r"..."`.
- En los ejercicios 1-6 se comprueba la cadena completa con `re.fullmatch(REn, texto)`. No basta con encontrar una parte válida dentro de ella.
- En los ejercicios 5 y 6 se comprueban también los grupos de captura, en el orden indicado y sin grupos adicionales. Usa `(?:...)` cuando necesites agrupar sin capturar.
- En los ejercicios 7 y 8 se evalúa `re.sub(REn, SUBn, texto)`, con sus opciones por defecto. No uses funciones de sustitución, bucles ni procesamiento adicional.
- Salvo en el ejercicio 7, los formatos no admiten espacios, tabuladores ni saltos de línea, excepto el espacio indicado expresamente en el ejercicio 8. Las letras son ASCII (a-z) y las cifras son ASCII (0-9).
- En los ejercicios 1 y 2 utiliza concatenación, alternativas y repeticiones; no uses aserciones de anticipación o retrospcción (`lookaround`).

Ejecuta las pruebas con `python3 -m unittest -v test_p1`. Los ejemplos facilitados no son exhaustivos: añade al menos cuatro casos nuevos por ejercicio (1-8), incluyendo casos límite y casos que deban rechazarse o permanecer sin cambios, según corresponda.

Ejercicio 0 · Ejemplo resuelto · Sin puntuación

Reconoce las cadenas no vacías sobre $\{a,b\}$ que empiezan por `a`.

```
RE0 = r"a[ab]*"
re.fullmatch(RE0, "abba") is not None # True
re.fullmatch(RE0, "ba") is not None   # False
```

Entrega: un único miembro de la pareja entregará `regular_expressions.py` y `test_p1.py` en Moodle. Incluye los nombres de ambos en un comentario y explica brevemente la idea de cada patrón. Los ocho ejercicios suman 10 puntos; se valorarán la corrección, los grupos o sustituciones exigidos y los casos límite.

Consulta: [documentación oficial del módulo re](#), especialmente `fullmatch`, grupos y `sub`.

Ejercicio 1 · Un número impar de ceros · 1 punto

Diseña RE1 sobre el alfabeto $\{0,1\}$ para reconocer las cadenas que contienen un **número impar de ceros**. Los unos pueden aparecer en cualquier posición y cantidad. La cadena vacía no pertenece al lenguaje.

Válidas: "0", "10", "000", "1010101".

No válidas: "", "1", "00", "010", "10a".

Ejercicio 2 · Unos separados · 1 punto

Diseña RE2 sobre el alfabeto $\{0,1\}$ para reconocer las cadenas en las que **no aparecen dos unos consecutivos**. Puede haber cualquier número de ceros consecutivos. Se admite la cadena vacía.

Válidas: "", "0", "1", "00100", "10101".

No válidas: "11", "0110", "1011", "102".

Ejercicio 3 · Importes con coma decimal · 1,5 puntos

Diseña RE3 para reconocer importes enteros o decimales con estas reglas:

- El signo + o - es opcional.
- La parte entera contiene una o más cifras. No admite ceros iniciales, salvo el propio 0.
- La parte decimal es opcional. Si aparece, consiste en una coma seguida de **exactamente dos cifras**.
- No se admiten separadores de miles, símbolos monetarios ni notación científica.

Válidas: "0", "-12", "+3,14", "42,00", "-0,50".

No válidas: "00", "012,30", "3,", "3,1", "3,141", "3.14".

Ejercicio 4 · Nombres de archivos de entrega · 1,5 puntos

Diseña RE4 para reconocer nombres con la forma nombre_apellido.txt o nombre_apellido.csv, opcionalmente precedidos por el directorio literal datos/.

- nombre y apellido son secuencias no vacías de letras minúsculas a-z.
- Debe aparecer exactamente un guion bajo entre ambos.
- Las únicas extensiones son .txt y .csv, en minúsculas.
- No se admiten otros directorios, espacios ni componentes adicionales.

Válidas: "ana_lopez.txt", "datos/luis_gil.csv", "a_b.csv".

No válidas: "Ana_lopez.txt", "ana__lopez.txt", "ana_lopez.pdf", "datos/ana_lopezXcsv", "otros/ana_lopez.csv".

Ejercicio 5 · Hora con grupos de captura · 1,5 puntos

Diseña RE5 para reconocer horas con el formato exacto hh:mm:ss y capturar por separado la hora, los minutos y los segundos.

- La hora está entre 00 y 23, ambos incluidos.
- Los minutos y los segundos están entre 00 y 59, ambos incluidos.
- Cada componente tiene exactamente dos cifras, incluidos los ceros iniciales necesarios.
- Los separadores son dos puntos literales. No se admiten zonas horarias ni segundos intercalares.
- Debe haber exactamente tres grupos: **1: hora; 2: minutos; 3: segundos**.

Válidas: "00:00:00", "09:07:05", "23:59:59".

No válidas: "24:00:00", "12:60:00", "12:00:60", "9:07:05", "09:07".

Para "09:07:05", `match.groups()` debe devolver ('09', '07', '05').

```
match = re.fullmatch(RE5, "09:07:05")
if match is not None:
    print(match.group(1)) # 09
    print(match.groups()) # ('09', '07', '05')
```

Ejercicio 6 · Colores RGB · 1,5 puntos

Diseña RE6 para reconocer colores con el formato exacto rgb(r,g,b) y capturar sus tres componentes numéricos. Las letras rgb y los paréntesis forman parte del texto.

- Cada componente representa un entero entre 0 y 255, ambos incluidos.
- No se admiten ceros iniciales, salvo el propio 0, ni signos.
- Las componentes se separan mediante comas, sin espacios.
- Debe haber exactamente tres grupos: **1: rojo; 2: verde; 3: azul**.

Válidas: "rgb(0,0,0)", "rgb(255,128,7)", "rgb(10,20,255)".

No válidas: "rgb(256,0,0)", "rgb(01,2,3)", "rgb(-1,2,3)", "rgb(1, 2,3)", "RGB(1,2,3)", "rgb(1,2,3,4)".

Para "rgb(255,128,7)", `match.groups()` debe devolver ('255', '128', '7').

Pista: expresa el intervalo numérico mediante alternativas según el número de cifras. Puedes reutilizar un mismo fragmento de patrón para las tres componentes. Los paréntesis literales del texto y los paréntesis de captura cumplen funciones distintas.

Sustituciones con expresiones regulares

`re.sub(patron, sustitucion, texto)` sustituye todas las coincidencias y devuelve el texto resultante. Si no hay coincidencias, devuelve el texto sin cambios. Una sustitución puede ser texto fijo o reutilizar grupos mediante `\g<1>`, `\g<2>`, etc.

Ejemplo ajeno a los ejercicios: intercambiar dos bloques de cifras.

```
re.sub(r"([0-9]+):([0-9]+)", r"\g<2>/\g<1>", "12:34")
```

Resultado: '34/12'

Ejercicio 7 · Normalizar espacios horizontales · 0,75 puntos

Completa RE7 y SUB7 para sustituir cada secuencia no vacía de **espacios ASCII y/o tabuladores** por un único espacio ASCII.

- La sustitución se aplica en todo el texto, incluidas sus posiciones inicial y final.
- No elimines los espacios iniciales o finales: también se reducen a un espacio.
- Conserva los saltos de línea y cualquier otro carácter. La cadena vacía queda igual.

En estos ejemplos se usa la notación de cadenas de Python: `\t` representa un tabulador y `\n` un salto de línea real.

Entrada: 'uno dos\ttres'	Salida: 'uno dos tres'
Entrada: ' hola\t '	Salida: ' hola '
Entrada: 'a\n\t b'	Salida: 'a\n b'
Entrada: 'sin_cambios'	Salida: 'sin_cambios'
Entrada: ''	Salida: ''

Ejercicio 8 · Reordenar un nombre · 1,25 puntos

Completa RE8 y SUB8 para transformar una cadena completa con el formato apellido, nombre en nombre apellido.

- Cada campo es una única palabra no vacía formada por letras a-z.
- Entre ambos campos hay exactamente una coma y un espacio ASCII.
- RE8 debe tener exactamente dos grupos: **1: apellido; 2: nombre**. SUB8 debe reutilizarlos en orden inverso, separados por un espacio.
- Si la cadena completa no cumple el formato, debe quedar intacta. No transformes fragmentos dentro de textos más largos.

Entrada: 'lopez, ana'	Salida: 'ana lopez'
Entrada: 'gil, luis'	Salida: 'luis gil'
Entrada: 'lopez,ana'	Salida: 'lopez,ana'
Entrada: 'lopez, ana'	Salida: 'lopez, ana'
Entrada: 'x lopez, ana'	Salida: 'x lopez, ana'
Entrada: 'lopez, ana\n'	Salida: 'lopez, ana\n'

Pista: `re.sub` busca coincidencias dentro del texto. Para exigir sus límites absolutos puedes utilizar `\A` y `\Z`. No uses `re.fullmatch` ni una condición adicional para realizar esta sustitución.